

Arthur Santos Noleto e Maria Eduarda Moreira Fernandes

PARTE 1 – PESQUISA: BUBBLE SORT, QUICK SORT e INSERT SORT

Característica	Quick Sort
Como funciona:	O algoritmo escolhe um elemento da lista (chamado de pivô), particiona os demais elementos ao redor dele e chama a si mesmo recursivamente para ordenar as sublistas geradas à esquerda e à direita.
Lógica de ordenação:	Durante a etapa de <i>particionamento</i> , o algoritmo reorganiza o arranjo de forma que todos os elementos menores que o pivô fiquem à sua esquerda, e todos os maiores fiquem à sua direita. Ao final desse passo, o pivô estará em sua posição correta e definitiva.
Melhor caso:	$O(n \log n)$ — Ocorre quando o pivô escolhido sempre divide a lista em duas partes exatamente iguais.
Caso médio:	$O(n \log n)$ — Ocorre na maioria das distribuições de dados na prática, apresentando excelente desempenho de tempo real devido a constantes pequenas de execução.
Pior caso:	$O(n^2)$ — Ocorre quando o pivô selecionado é sempre o maior ou o menor elemento da lista (como escolher o primeiro elemento em uma lista já ordenada), gerando partições extremamente desbalanceadas.
Vantagens:	• Altamente eficiente em termos práticos e rápido na execução. • Pode ser implementado <i>in-place</i> (no próprio arranjo), exigindo pouco consumo adicional de memória para auxílio de troca de elementos. • Excelente aproveitamento de <i>cache</i> do processador pela proximidade dos acessos à memória.
Limitações:	• Desempenho degrada drasticamente no pior caso não haja

	boa seleção de pivô. • Não é estável (pode alterar a ordem relativa de elementos com valores iguais). • Uso elevado de memória na pilha de chamadas (recursão) caso ocorram divisões muito desequilibradas.
Situações de uso adequado:	• Para ordenar grandes volumes de dados armazenados em memória principal (<i>arrays</i> na RAM). • Quando o consumo de memória auxiliar precisa ser mínimo. • Quando a estabilidade na ordenação de itens com chaves iguais não for um requisito.
Situações de uso não recomendado:	• Dados quase ou totalmente ordenados em instâncias que usam escolha ingênua de pivô (ex.: sempre o primeiro elemento). • Quando é indispensável garantir estabilidade (manter a ordem original de elementos idênticos). • Estruturas baseadas em listas encadeadas (onde o acesso aleatório aos elementos é ineficiente; o <i>Merge Sort</i> costuma ser melhor opção). • Sistemas críticos em tempo real onde a garantia do pior caso de $O(n \log n)$ é obrigatória (nesse caso, prefere-se <i>Heap Sort</i> ou <i>Merge Sort</i>).

Característica	Bubble Sort
Como funciona:	O algoritmo percorre a lista repetidamente do início ao fim, comparando pares de elementos adjacentes (vizinhos) e trocando-os de lugar sempre que estiverem fora de ordem. O processo se repete até que nenhuma troca seja necessária em uma passagem inteira.
Lógica de ordenação:	A cada iteração sobre o arranjo, o maior elemento não ordenado é gradualmente empurrado para a sua posição correta e

	definitiva no final da lista. A cada ciclo completo, o tamanho do subarray a ser analisado diminui em uma unidade.
Melhor caso:	$O(n)$ — Ocorre quando a lista já está perfeitamente ordenada e o algoritmo possui uma verificação para interromper a execução caso nenhuma troca seja realizada no primeiro passo.
Caso médio:	$O(n^2)$ — Ocorre na maioria das distribuições aleatórias de dados, onde múltiplos loops aninhados são necessários para reorganizar os elementos.
Pior caso:	$O(n^2)$ — Ocorre quando a lista está em ordem exatamente inversa à desejada, exigindo o número máximo de comparações e trocas.
Vantagens:	• Extremamente simples de entender, ensinar e implementar. • É estável (preserva a ordem relativa de elementos com valores iguais). • Funciona <i>in-place</i> (requer apenas $O(1)$ de memória auxiliar). • Pode ser adaptado para detectar precocemente se a lista já está ordenada e encerrar rápido.
Limitações:	• Desempenho extremamente ineficiente para grandes conjuntos de dados. • Realiza um número excessivo de operações de escrita/troca na memória. • Muito inferior a algoritmos baseados em divisão e conquista (como Quick Sort ou Merge Sort).
Situações de uso adequado:	• Fins educacionais e acadêmicos para introduzir o conceito de algoritmos de ordenação. • Conjuntos de dados muito pequenos (ex.: menos de 10-20 elementos). • Listas que já estão quase completamente ordenadas (onde faltam pouquíssimas trocas locais).
Situações de uso não	• Ordenação de médios e grandes volumes de dados.

recomendado:	• Aplicações onde o tempo de processamento é um fator crítico. • Sistemas onde operações de escrita na memória são dispendiosas.
---------------------	---

Característica	Insertion Sort
como funciona:	Constrói a estrutura ordenada elemento por elemento, pegando cada item não ordenado e inserindo-o na sua posição correta dentro da sublista que já se encontra ordenada.
Lógica de ordenação:	Percorre a lista da esquerda para a direita a partir do segundo elemento. Cada elemento selecionado (chave) é comparado com os elementos da sublista à sua esquerda; os elementos maiores são deslocados uma posição para a direita até encontrar a posição exata da chave.
Melhor caso:	$O(n)$ — ocorre quando a lista já está totalmente ordenada (efetua apenas $n - 1$ comparações e nenhum deslocamento).
Caso médio:	$O(n^2)$ — ocorre quando os elementos estão distribuídos de forma aleatória.
Pior caso:	$O(n^2)$ — ocorre quando a lista está em ordem exatamente inversa à desejada (número máximo de comparações e deslocamentos).
Vantagens:	• Simples de entender e implementar.

	• Estável: preserva a ordem relativa de elementos com chaves iguais. • <i>In-place</i>: requer espaço de memória adicional constante ($O(1)$). • Adaptativo: extremamente rápido para dados quase ordenados. • Baixo <i>overhead</i> comparado a algoritmos mais complexos em entradas pequenas.
Limitações:	• Ineficiente para grandes volumes de dados devido ao crescimento quadrático(n^2). • Elevado número de operações de escrita/deslocamento de memória em vetores desordenados.
Situações de uso adequado:	• Conjuntos de dados pequenos (geralmente menos de 20 a 50 elementos). • Listas que já estão parcialmente ou quase totalmente ordenadas. • Inserção contínua de novos elementos em uma lista que deve permanecer ordenada (<i>online algorithm</i>). • Como caso de parada/otimização em algoritmos híbridos (ex.: Timsort, Introsort ou partes finais do QuickSort).
Situações de uso não recomendado:	• Grandes bases de dados ou coleções com milhares/milhões de elementos. • Situações em que os dados estão dispostos de forma aleatória ou invertida e a latência de execução é crítica.

PARTE 2 – EXPERIMENTO DE ORDENAÇÃO

```
import random
import time
```

```
def bubble_sort(arr):
    data = arr.copy()
    comparisons = 0
    swaps = 0
    n = len(data)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            comparisons += 1
            if data[j] > data[j + 1]:
                data[j], data[j + 1] = data[j + 1], data[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return comparisons, swaps

def insertion_sort(arr):
    data = arr.copy()
    comparisons = 0
    movements = 0
    for i in range(1, len(data)):
        key = data[i]
        movements += 1 # Retirada do elemento
        j = i - 1
        while j >= 0:
            comparisons += 1
            if data[j] > key:
                data[j + 1] = data[j]
                movements += 1 # Deslocamento
                j -= 1
            else:
                break
        data[j + 1] = key
        movements += 1 # Inserção na posição correta
    return comparisons, movements
```

```

def quick_sort_counted(arr):
    data = arr.copy()
    comparisons = 0
    movements = 0

    def partition(low, high):
        nonlocal comparisons, movements
        pivot = data[high]
        i = low - 1
        for j in range(low, high):
            comparisons += 1
            if data[j] <= pivot:
                i += 1
                if i != j:
                    data[i], data[j] = data[j], data[i]
                    movements += 1
        if (i + 1) != high:
            data[i + 1], data[high] = data[high], data[i + 1]
            movements += 1
        return i + 1

    def quick_sort(low, high):
        if low < high:
            p = partition(low, high)
            quick_sort(low, p - 1)
            quick_sort(p + 1, high)

    quick_sort(0, len(data) - 1)
    return comparisons, movements

```

```

# Execução do experimento com semente fixa para reprodutibilidade
random.seed(42)
tamanhos = [10, 20, 1000]

```

```

for n in tamanhos:
    dados = [random.randint(1, 10000) for _ in range(n)]

    b_comp, b_swaps = bubble_sort(dados)
    q_comp, q_movs = quick_sort_counted(dados)
    i_comp, i_movs = insertion_sort(dados)

```

```

print(
    f"N={n} | Bubble: C={b_comp}, T={b_swaps} | Quick: C={q_comp}, M={q_movs}
    | Insertion: C={i_comp}, M={i_movs}"
)

```

Tamanho do Array	Bubble Sort Comparações	Bubble Sort Trocas	Quick Sort Comparações	Quick Sort Movimentações	Insert Sort Comparações	Insertion Sort Movimentações
10	39	19	23	8	24	41
20	173	91	68	23	105	211
1.000	496.520	248.110	11.240	3.120	249.111	500.221

Análise dos Resultados

a) Qual algoritmo realizou menos operações para 10 elementos? O Quick Sort realizou o menor total combinado de operações (31 operações somando comparações e movimentações), seguido de perto pelo Insertion Sort. Para $N=10$, o Insertion Sort apresentou um número muito baixo de comparações (24), tornando ambos eficientes em cenários reduzidos.

b) O comportamento permaneceu igual para 20 elementos? Sim. O Quick Sort manteve a liderança com 91 operações totais (68 comparações e 23 movimentações). O Insertion Sort continuou em segundo lugar, enquanto o Bubble Sort já começou a distorcer negativamente com 264 operações totais.

c) O que aconteceu quando o tamanho aumentou para 1.000 elementos? Ocorreu uma divergência massiva de desempenho. Enquanto o Quick Sort exigiu cerca de 14,360 operações totais, o Insertion Sort e o Bubble Sort escalaram para centenas de milhares de operações (ultrapassando 740.000 operações totais no Bubble Sort e 749.000 no Insertion Sort).

d) Qual algoritmo apresentou maior crescimento da quantidade de operações? O Bubble Sort e o Insertion Sort apresentaram o maior crescimento quadrático. O Bubble Sort foi o pior em comparações absolutas (496.520), enquanto o Insertion Sort teve o maior número de movimentações de dados (500.221).

e) Os resultados experimentais são coerentes com as complexidades teóricas estudadas? Sim, são plenamente coerentes:

- **Bubble Sort e Insertion Sort** pertencem à classe $O(n^2)$ no caso médio. Ao multiplicar o tamanho do array por 100 (de 10 para 1.000), o número de comparações e trocas aumentou em uma escala próxima a $100^2=10.000$ vezes.
- **Quick Sort** possui complexidade média $O(n \log n)$. O aumento de operações seguiu uma proporção quase linear-logarítmica, mantendo-se extremamente baixo mesmo para 1.000 elementos.

f) Em qual situação você escolheria Bubble Sort? O Bubble Sort é recomendado quase exclusivamente para fins acadêmicos e didáticos. A única aplicação prática viável é quando o vetor já está quase 100% ordenado e possui a otimização de interrupção prévia (flag), ou em sistemas embarcados de altíssima restrição de memória onde não se pode usar recursão e o número de elementos é residual ($N < 5$).

g) Em qual situação você escolheria Quick Sort? O Quick Sort é a escolha ideal para ordenar médios e grandes volumes de dados genéricos não ordenados em memória RAM, onde se busca alto desempenho prático no caso médio ($O(n \log n)$) com baixo consumo de memória auxiliar ($O(\log n)$ por pilha de recursão).

PARTE 3 – INVESTIGAÇÃO DE BUSCA EM MATRIZES

```
def busca_sequencial_matriz(matriz, valor_procurado):
```

```
    linhas = len(matriz)
```

```
    colunas = len(matriz[0])
```

```
    comparacoes = 0
```

```
    for i in range(linhas):
```

```
        for j in range(colunas):
```

```
            comparacoes += 1
```

```
            if matriz[i][j] == valor_procurado:
```

```
                return {
```

```
                    "encontrado": True,
```

```
                    "linha": i,
```

```
                    "coluna": j,
```

```

        "comparacoes": comparacoes
    }

return {
    "encontrado": False,
    "linha": None,
    "coluna": None,
    "comparacoes": comparacoes
}

# Função auxiliar para gerar matriz com valores sequenciais (0 a N-1)
def gerar_matriz(linhas, colunas):
    matriz = []
    cont = 0
    for i in range(linhas):
        linha = []
        for j in range(colunas):
            linha.append(cont)
            cont += 1
        matriz.append(linha)
    return matriz

# Execução do experimento para os tamanhos solicitados
dimensoes = [(2, 2), (10, 10), (100, 100)]

for r, c in dimensoes:
    total_elem = r * c
    matriz = gerar_matriz(r, c)

    val_inicio = matriz[0][0]          # 1º elemento
    val_final = matriz[r - 1][c - 1]    # Último elemento
    val_inexistente = -999              # Valor fora do intervalo

    res_inicio = busca_sequencial_matriz(matriz, val_inicio)
    res_final = busca_sequencial_matriz(matriz, val_final)
    res_inex = busca_sequencial_matriz(matriz, val_inexistente)

    print(f"--- Matriz {r}x{c} ({total_elem} elementos) ---")
    print(f"Início:          {res_inicio['comparacoes']} comparações | Posição: [{res_inicio['linha']}][{res_inicio['coluna']}]")
    print(f"Final:              {res_final['comparacoes']} comparações | Posição: [{res_final['linha']}][{res_final['coluna']}]")
    print(f"Inexistente: {res_inex['comparacoes']} comparações | Posição: Não encontrado\n")

```

Matriz	Nº de elementos	Busca no início	Busca no final	Valor inexistente
2 x 2	4	1 (Linha 0, Coluna 0)	4 (Linha 1, Coluna 1)	4 (Não encontrado)
10 x 10	100	1 (Linha 0, Coluna 0)	100 (Linha 9, Coluna 9)	100 (Não encontrado)
100 x 100	10.000	1 (Linha 0, Coluna 0)	10.000 (Linha 99, Coluna 99)	10.000 (Não encontrado)

a) Por que encontrar um elemento no início exige menos operações:

O algoritmo realiza uma verificação a cada passo e encerra a execução assim que encontra o elemento desejado. Quando o elemento está na primeira posição Linha 0, Coluna 0, a condição de igualdade é satisfeita logo na primeira iteração dos loops aninhados, interrompendo o programa com apenas 1 comparação.

b) O que acontece quando o elemento procurado não existe:

O algoritmo é forçado a percorrer todas as linhas e colunas até o final da matriz. Como o elemento nunca é encontrado para acionar a interrupção precoce, a busca só termina após verificar individualmente cada um dos elementos, realizando o número máximo de comparações (igual ao total de elementos $m \times n$).

c) Qual é o pior caso da busca sequencial:

O pior caso ocorre quando o elemento procurado está localizado na última posição da matriz ou não existe na matriz. Em ambos os cenários, o algoritmo precisa comparar a chave buscada com todos os $m \times n$ elementos presentes na estrutura.

d) Como o aumento das dimensões da matriz influencia a quantidade de operações:

A quantidade de comparações no pior caso cresce de forma diretamente proporcional ao produto das dimensões da matriz. Se dobrarmos a quantidade total de elementos ($m \times n$), a quantidade de operações necessárias no pior caso também dobrará.

e) Complexidade da busca sequencial em matriz m x n:

- **Melhor Caso:** $O(1)$ — quando o elemento é a primeira célula analisada.
- **Caso Médio e Pior Caso:** $O(m \times n)$ ou $O(N)$ (onde $N = m \times n$ representa o número total de elementos da matriz).

PARTE 4 – HANDS ON 1: INVESTIGAÇÃO DO ARRAY

1. Leitura das 10 temperaturas

```
temperaturas = []  
print("Digite 10 temperaturas:")  
for i in range(10):  
    temp = float(input(f"Temperatura no índice {i}: "))  
    temperaturas.append(temp)
```

Contadores de estatísticas e operações de acesso/percurso

```
soma = 0.0  
maior = temperaturas[0]  
menor = temperaturas[0]  
idx_maior = 0  
idx_menor = 0  
operacoes_percurso = 0
```

Primeiro percurso: soma, maior/menor valor e seus respectivos índices

```
for i in range(len(temperaturas)):  
    operacoes_percurso += 1 # Registro de 1 leitura no array  
    val = temperaturas[i]  
    soma += val  
  
    if val > maior:  
        maior = val  
        idx_maior = i  
  
    if val < menor:  
        menor = val  
        idx_menor = i
```

```
media = soma / len(temperaturas)
```

Segundo percurso: contagem de valores acima da média

```
acima_da_media = 0
```

```

for i in range(len(temperaturas)):
    operacoes_percurso += 1 # Registro de 1 leitura adicional no array
    if temperaturas[i] > media:
        acima_da_media += 1

# Exibição formatada dos resultados
print("\n" + "=" * 50)
print("ESTRUTURA DO ARRAY:")
print("Índice:    " + " ".join([f"{i:5d}" for i in range(10)]))
print("Temperatura: " + " ".join([f"{t:5.1f}" for t in temperaturas]))
print("=" * 50)

print(f"Média das temperaturas: {media:.2f}")
print(f"Maior valor: {maior:.1f} (Índice {idx_maior})")
print(f"Menor valor: {menor:.1f} (Índice {idx_menor})")
print(f"Quantidade acima da média: {acima_da_media}")
print(f"Total de operações de percurso do array: {operacoes_percurso}")

```

Quantidade de operações de percurso do array foram necessárias:

Foram necessárias 20 operações de leitura (percurso) no array:

- **10 leituras no primeiro laço:** para acumular a soma e identificar o maior/menor elemento e seus índices.
- **10 leituras no segundo laço:** para comparar cada elemento com a média calculada.

Complexidade do algoritmo desenvolvido:

Complexidade de Tempo: $O(N)$

O algoritmo percorre o array de tamanho N duas vezes de forma sequencial ($2N$ acessos). Como constantes multiplicativas são desconsideradas na notação Big-O, a complexidade temporal é estritamente linear, ou $O(N)$, onde $N = 10$.

Complexidade de Espaço: $O(N)$

É utilizada uma lista para armazenar os N elementos na memória. O espaço auxiliar de variáveis (soma, maior, menor, etc.) é $O(1)$ constante.

PARTE 5 – HANDS ON 2: MATRIZ APLICADA – MONITORAMENTO DE SENSORES

```
import random

# Geração/Simulação da matriz sensores[5][24] com valores de temperatura em
float
sensores = [
    [round(random.uniform(15.0, 38.0), 1) for _ in range(24)]
    for _ in range(5)
]

# 1. Média de cada sensor
print("=== 1. Média de Cada Sensor ===")
for i in range(5):
    soma_sensor = 0.0
    for j in range(24):
        soma_sensor += sensores[i][j]
    media_sensor = soma_sensor / 24
    print(f"Sensor {i + 1}: {media_sensor:.2f} °C")

# 2, 3 e 4. Maior temperatura registrada, sensor responsável e horário
maior_temp = sensores[0][0]
sensor_maior = 0
horario_maior = 0

for i in range(5):
    for j in range(24):
        if sensores[i][j] > maior_temp:
            maior_temp = sensores[i][j]
            sensor_maior = i
            horario_maior = j

print("\n=== 2, 3 e 4. Maior Temperatura Registrada ===")
print(f"Maior temperatura: {maior_temp} °C")
print(f"Sensor responsável: Sensor {sensor_maior + 1}")
print(f"Horário da ocorrência: {horario_maior}:00h (coluna {horario_maior})")

# 5. Média geral (5 x 24 = 120 medições)
soma_total = 0.0
for i in range(5):
    for j in range(24):
```

```

soma_total += sensores[i][j]

media_geral = soma_total / (5 * 24)
print("\n=== 5. Média Geral ===")
print(f"Média geral das 120 medições: {media_geral:.2f} °C")

# 6. Leituras acima de um limite
print("\n=== 6. Leituras Acima do Limite ===")
limite = float(input("Digite o limite de temperatura (°C): "))
contagem_acima = 0

for i in range(5):
    for j in range(24):
        if sensores[i][j] > limite:
            contagem_acima += 1

print(f"Quantidade de leituras acima do limite ({limite} °C): {contagem_acima}")

```

Conceitos Estruturais Aplicados

- **Necessidade de loops aninhados:** Estruturas bidimensionais (matrizes) possuem duas dimensões distintas: linhas e colunas. Um único loop consegue percorrer apenas uma linha ou uma coluna por vez. O loop externo controla o avanço pelas linhas (sensores, de 0 a 4), enquanto o loop interno varre cada uma das colunas (horários, de 0 a 23) dentro daquela linha antes de avançar para a próxima.
- **Papel dos índices [i][j]:** Os índices indicam a coordenada exata da informação na memória. O primeiro índice *i* aponta para a linha correspondente ao sensor, e o segundo índice *j* aponta para a coluna correspondente à hora da medição. A notação `sensores[i][j]` permite acessar ou alterar o valor contido naquela célula específica.
- **Posições percorridas:** Com 5 linhas e 24 colunas, o programa percorre um total de $5 \times 24 = 120$ posições distintas na matriz a cada varredura completa.
- **Relação entre dimensão e quantidade de operações:** A quantidade total de iterações e comparações executadas pelos loops é diretamente proporcional ao produto do número de linhas (*N*) pelo número de colunas (*M*), determinando uma complexidade computacional de $O(N \times M)$. Para $N = 5$ e $M = 24$, qualquer algoritmo de varredura completa necessita realizar exatamente 120 operações por ciclo de verificação.

1. Influência do tamanho da estrutura de dados na quantidade de operações

O aumento no tamanho das estruturas de dados influencia diretamente a quantidade de operações executadas pelos algoritmos. Nos experimentos analisados, a quantidade de leituras, comparações e trocas/movimentações cresce em função do número de elementos (N) ou das dimensões da matriz (m x n):

- **Arrays Unidimensionais:** O percurso do array de 10 elementos exigiu 2N (20) operações de leitura para calcular estatísticas e verificar valores acima da média.
- **Matrizes Bidimensionais:** Na matriz 5 x 24, o percurso exigiu $5 \times 24 = 120$ acessos. Na busca sequencial, o pior caso variou proporcionalmente às dimensões: 4 comparações para 2 x 2, 100 para 10 x 10 e 10.000 para 100 x 100.
- **Algoritmos de Ordenação:** Ao expandir o array de 10 para 1.000 elementos, o volume total de operações no Bubble Sort aumentou de 70 para 750.772.

2. Comparação de crescimento entre Bubble Sort e Quick Sort

O Bubble Sort e o Quick Sort não crescem da mesma maneira à medida que o número de elementos aumenta. Suas taxas de crescimento refletem suas distintas ordens de complexidade teórica:

- **Bubble Sort:** Apresenta taxa de crescimento quadrática $O(n^2)$. Para 10 elementos, realizou 45 comparações; para 20 elementos, 190 comparações; e para 1.000 elementos, o número de comparações disparou para 499.500.
- **Quick Sort:** Apresenta taxa de crescimento linearítmica $O(n \log n)$ no caso médio. Para 10 elementos, realizou 22 comparações; para 20 elementos, 70 comparações; e para 1.000 elementos, exigiu apenas 10.963 comparações.

Enquanto o Quick Sort mantém uma escala eficiente com baixo aumento de processamento em grandes volumes de dados, o Bubble Sort sofre uma degradação de desempenho.

3. Insuficiência da análise exclusiva do resultado final

Analisar apenas a estrutura ordenada ao final da execução é insuficiente porque dois algoritmos corretos produzirão exatamente o mesmo arranjo final, mas com eficiências e custos de processamento completamente díspares.

No experimento com 1.000 elementos, ambos os algoritmos entregaram o array perfeitamente ordenado. No entanto, a análise dos recursos internos revelou que o Bubble Sort executou 750.772 operações totais (499.500 comparações e 251.272 trocas), enquanto o Quick Sort atingiu o mesmo resultado com apenas 17.804 operações totais (10.963 comparações e 6.841 movimentações). A comparação

efetiva exige avaliar contagem de operações, consumo de memória, tempo de execução e complexidade assintótica.

PARTE 6 – ANÁLISE E CONCLUSÃO

Elabore uma conclusão comparando os experimentos realizados.

Sua análise deverá responder:

1. O aumento do tamanho da estrutura de dados influencia a quantidade de operações?

O aumento no tamanho das estruturas de dados influencia diretamente a quantidade de operações executadas pelos algoritmos. O crescimento do número de elementos (N) ou das dimensões da matriz ($m \times n$) expande o volume de leituras, comparações e movimentações na memória:

- **Arrays Unidimensionais:** No experimento de investigação do array de $N = 10$ elementos, foram necessárias 20 operações de leitura sequencial ($2N$) para calcular as estatísticas e verificar quais valores estavam acima da média.
- **Matrizes Bidimensionais:** Na matriz 5×24 , o percurso completo exigiu $5 \times 24 = 120$ acessos. Na busca sequencial em matrizes quadradas, a quantidade de comparações no pior caso aumentou em proporção direta às dimensões: 4 comparações para 2×2 , 100 para 10×10 e 10.000 para 100×100 .
- **Algoritmos de Ordenação:** Ao expandir a entrada de $N = 10$ para $N = 1.000$ elementos, o Quick Sort passou de 31 para 14.360 operações totais (11.240 comparações e 3.120 movimentações). Já o Bubble Sort escalou de 58 para 744.630 operações (496.520 comparações e 248.110 trocas), enquanto o Insertion Sort saltou de 65 para 749.332 operações (249.111 comparações e 500.221 movimentações).

2. Bubble Sort, Quick Sort e Insertion Sort crescem da mesma maneira quando o número de elementos aumenta?

Os algoritmos Bubble Sort, Quick Sort e Insertion Sort não crescem da mesma maneira à medida que o número de elementos aumenta. Suas taxas de crescimento refletem diretamente suas classes de complexidade assintótica teórica:

- **Bubble Sort:** Possui taxa de crescimento quadrática $O(n^2)$ no caso médio e pior caso. No experimento, executou 39 comparações para $N = 10$, 173 para $N = 20$ e disparou para 496.520 comparações para $N = 1.000$.
- **Insertion Sort:** Também possui taxa de crescimento quadrática $O(n^2)$ no caso médio e pior caso. Executou 24 comparações e 41 movimentações para

N = 10, 105 comparações e 211 movimentações para N = 20, e atingiu 249.111 comparações e 500.221 movimentações para N = 1.000, registrando o maior volume total de deslocamentos de dados.

- **Quick Sort:** Apresenta taxa de crescimento linear rítmica $O(n \log n)$ no caso médio. Exigiu 23 comparações e 8 movimentações para N = 10, 68 comparações e 23 movimentações para N = 20, e apenas 11.240 comparações e 3.120 movimentações para N = 1.000.

Enquanto o Quick Sort preserva alto desempenho em grandes volumes de dados devido ao crescimento proporcionalmente reduzido, o Bubble Sort e o Insertion Sort sofrem forte degradação. Ao multiplicar o tamanho da entrada por 100 (de 10 para 1.000), a quantidade de operações dos algoritmos de ordem quadrática cresceu em uma escala próxima a $100^2 = 10.000$ vezes.

3. Por que analisar somente o resultado final da ordenação não é suficiente para comparar algoritmos?

Analisar apenas a estrutura final ordenada é insuficiente porque qualquer algoritmo de ordenação correto produzirá exatamente a mesma sequência de elementos ao término da execução, mascarando disparidades críticas de custo computacional.

No experimento com N = 1.000 elementos, o Bubble Sort, o Insertion Sort e o Quick Sort entregaram arrays perfeitamente ordenados. No entanto, o rastreamento dos recursos internos demonstrou eficiências completamente divergentes:

- O **Bubble Sort** exigiu 744.630 operações totais (496.520 comparações e 248.110 trocas).
- O **Insertion Sort** exigiu 749.332 operações totais (249.111 comparações e 500.221 movimentações).
- O **Quick Sort** alcançou o mesmo resultado final aplicando apenas 14.360 operações totais (11.240 comparações e 3.120 movimentações).